The Prompt Is Not the Program

Correction Genealogies as Executable Specifications

Watson Hartsoe

Working paper · 17 August 2026
Slipcase checkpoint SLIPCASE-20260817-203946

AI-augmented working paper. The assembly instrument, source map, and making history are preserved in the accompanying Slipcase package.

Abstract

Natural-language interaction with generative models is often described either as a new programming interface or as a form of instruction writing. Both descriptions put too much weight on the prompt as a finished textual object. Early prompt communities already exhibited a different practice: users generated, inspected, selected, varied, compared, and repeated, often discovering what they wanted only through the behavior of the system. This paper argues that the more consequential unit is a *correction genealogy*: a stateful lineage in which descriptions, outputs, judgments, counterexamples, invariants, and revisions accumulate into an executable specification. The claim is deliberately narrower than "prompting is new programming." Reflective design predates generative AI; program synthesis already supports partial programs; oracle-guided synthesis already learns from counterexamples; interactive evolutionary computation already places human judgment inside a search loop. What changes in contemporary generative practice is their conjunction with semantically underdetermined natural-language inputs that can execute before the user has represented all consequential variables. The resulting system can reveal unmarked holes in the specification, alter the evaluator's criteria, and turn failures into tests. Drawing from an early ethnography of Midjourney prompt craft and from work on program sketching, reflective design, interactive evolutionary computation, oracle-guided synthesis, delta debugging, metamorphic testing, property-based testing, active learning, and open-ended search, I develop a practical research program for treating prompt work as experimental specification building rather than prompt polishing. The central design implication is archival: preserving final prompts is insufficient. Systems should preserve the genealogy that explains why a prompt has the constraints it has and provide machinery that can replay, minimize, discriminate, and attack those constraints.

1 The wrong unit

The most visible artifact in contemporary generative work is the prompt. It is easy to copy, publish, compare, buy, teach, and blame. That visibility encourages a simple model: a user has an intention, writes an instruction, the model executes it, and better prompting consists of learning to write more effective instructions. In this account the prompt is analogous to source code, a command, or a compact specification.

An early ethnography of the Midjourney community already made this model difficult to sustain. Users described prompts as "chisels" applied to noise; expert practitioners reported generating thousands or tens of thousands of images around a promising phrase; some hoarded prompts as

trade secrets, while others maintained spreadsheets of randomized artists, styles, and nouns. The study's prompt-craft section summarizes the local pedagogy succinctly: trial and error is the teacher [1]. Those practices matter because they relocate expertise. A final phrase may be useful, but it is the residue of a longer activity: choose a move, observe what the system does, diagnose the difference, preserve or discard a branch, and make another move.

That activity is not adequately represented as

$$\text{prompt} \rightarrow \text{output}.$$

A more faithful minimal unit is

$$D_t \rightarrow A_t \rightarrow J_t \rightarrow \Delta_t \rightarrow D_{t+1},$$

where D_t is a description or partial specification, A_t the generated artifact, J_t a judgment, and Δ_t a revision. Yet even this loop is too conservative if it assumes that judgment is always performed against a fixed specification. In practice, the artifact can expose a variable the user had not represented, or cause the user to articulate a criterion that did not exist in explicit form before generation. The next state is therefore not merely a better instruction. It may contain a changed specification.

I call the preserved lineage of these changes a *correction genealogy*. The term emphasizes two points. First, a revision inherits from a concrete encounter with a prior artifact; it is not simply a new wording. Second, the lineage matters because later constraints become intelligible only in relation to the failures, surprises, counterexamples, and accepted properties that produced them. A correction genealogy is therefore not a chat transcript by another name. It is a structured record of evolving executable commitments.

This paper asks what becomes visible if the correction genealogy, rather than the prompt, is treated as the unit of natural-language programming practice. The answer is not that all earlier software concepts disappear. The opposite is more productive: older ideas become unusually precise instruments for decomposing what prompt practitioners are already doing.

2 Prompt craft as search, not incantation

The Midjourney study is useful because it captured a community before a stable professional vocabulary for prompt work had fully formed. Users relied on metaphors—magic words, mountains and valleys in "latent space," sculpting noise—alongside empirical routines. Joseph described a phrase that opened a useful aesthetic region and then explored it across more than 10,000 generations; Ignite maintained randomized generation protocols; Shambibble approached wording and parameters experimentally; Oscar probed stereotypes and biases [1]. The metaphors are technically unreliable in places, but the practices are legible as search.

Interactive evolutionary computation provides one older formal ancestor. Takagi defines a family of systems in which evolutionary search is coupled to subjective human evaluation when the desired objective is hard to encode directly [2]. The parallel is not genealogical: Midjourney users were not necessarily running evolutionary algorithms. But it identifies a mechanism that the language of "prompt skill" obscures. A user can contribute a fitness judgment without possessing an explicit formula for the desired artifact. They can say "more like this," "keep that," "not this," or simply choose a branch.

The distinction matters because it separates at least three loci of expertise:

1. **Description expertise**: producing a useful current instruction.
2. **Evaluation expertise**: recognizing a promising or failed output.
3. **Search-policy expertise**: deciding what to vary, preserve, branch, or revisit next.

A repository of final prompts preserves only the first reliably. It often erases the second and third.

Search also explains why more precise specification is not always better. Lehman and Stanley's novelty-search work shows that fixed objectives can be deceptive: candidates that look closer to a goal can lead into local optima, while necessary stepping stones may initially appear worse [3]. Picbreeder makes this social. Users can branch from images evolved by others, so an intermediate form can become valuable because of descendants the original creator never imagined [4]. The creative contribution becomes phylogenetic: an apparently mediocre node can have high descendant generativity.

For prompt practice, the consequence is severe. If every iteration is judged only by resemblance to the already imagined target, a user may systematically prune the outputs that would have revealed a better target. The correction genealogy should therefore preserve failed, strange, and abandoned branches when they changed the search state. This is not sentimentality about process. It is an empirical claim about reachability: a discarded artifact may contain information about regions of behavior that are otherwise lost.

Recent work makes the search picture still less comfortable. A VLM-driven replication of Picbreeder reports non-monotonic effects of context: too little history permits repetition, but more history does not monotonically improve open-ended exploration and can participate in repetitive attractors [5]. POET goes further by coevolving problems and solutions, allowing competence discovered for one environment to transfer into another [6]. These systems suggest two provocations for prompt work. Memory may require curation rather than accumulation; and the task itself may need to change during discovery.

Neither result licenses the slogan that "forgetting is always creative" or that every prompt should mutate its objective. The useful point is narrower: once prompting is understood as search, the selection rule, memory policy, and stability of the problem become first-class design variables. They are invisible if analysis stops at the wording of the final prompt.

3 What is not new

A defensible account of contemporary prompting must resist novelty by resemblance. Several features that feel new have clear predecessors.

3.1 Artifacts have long talked back

Schön described design as a reflective conversation with the materials of a situation: a practitioner makes a move, encounters its consequences, and reframes subsequent action [7]. This is already very close to the familiar generative loop of describe, render, inspect, and revise. Therefore the fact that an artifact can surprise its maker or alter a design decision is not, by itself, a novelty of generative AI.

The stronger distinction concerns the speed, range, and executability of underdetermined descriptions. A sentence can now produce a high-dimensional artifact quickly enough that previously tacit criteria become visible through repeated computation. The historical continuity with reflective design should be preserved, because it lets us ask a better question: what changes when the "material"

that talks back is a learned generative system capable of supplying many unspecified details on every execution?

3.2 Incomplete programs already execute through synthesis

Program sketching supplies an even sharper counterexample to claims that natural-language prompting invented deferred specification. Solar-Lezama's sketches are partial programs containing explicit holes; synthesis searches for completions satisfying constraints while preserving programmer-provided structure [8]. The important difference is not that prompts can be incomplete. It is that much of their incompleteness is *unmarked*.

Consider "make a house overlooking the ocean." The expression contains no explicit placeholders for camera position, building material, number of stories, roof form, weather, time of day, site slope, historical period, or occupancy. Yet rendering requires some values for many such variables. The model must effectively participate in two operations: infer which variables need completion, then choose values for them. The user can discover those variables only after seeing what the generator selected.

This suggests a distinction between an *explicit hole*, represented in the program as an unknown, and an *unmarked hole*, revealed only by execution. The distinction remains provisional because natural-language interpretation, defaults, and underdetermination are difficult to separate cleanly. But it is experimentally testable: compare a formally enumerated partial specification with a natural-language description of the same task and inventory consequential output decisions that were absent from the description.

3.3 Counterexample-guided synthesis already iterates

Oracle-guided inductive synthesis and counterexample-guided inductive synthesis likewise predate LLM prompting. Jha and Seshia formalize synthesis procedures that iteratively query an oracle and use examples or counterexamples to refine candidate programs [9]. Again, the resemblance is useful precisely because it reveals a difference.

In the clean formal picture, a candidate changes as the learner receives evidence from an oracle. In reflective prompt practice, the evaluator can change too. An output can make a user notice a previously unrepresented criterion. The next judgment is then made by a different effective evaluation function. Schematically,

$$(C_t, O_t) \to e_t \to (C_{t+1}, O_{t+1}),$$

where C_t is the candidate and O_t is the evaluator state. The candidate and the specification can coevolve.

Calling this a "mutating oracle" is a formalization, not a term from OGIS. It also risks romanticizing inconsistency. A human evaluator can change because of fatigue, anchoring, novelty seeking, or noise. The research problem is to distinguish specification discovery from preference drift. A correction genealogy makes that distinction investigable because it records when a criterion first appears and what artifact made it salient.

4 From corrections to executable specification

If the prompt is not the program, what should replace it? The strongest answer in this field is not a new syntax. It is a richer executable record.

A correction genealogy contains at least:

$$\{D_t, A_t, J_t, \Delta_t, R_t, I_t\}_{t=0}^{n},$$

where R_t records reasons or counterexamples and I_t records accepted invariants. The important shift is that these components can be operationalized. A failure can become a reducer; a relation can become a metamorphic test; an invariant can become a property generator; uncertainty can determine the next diagnostic prompt.

4.1 Minimize the failure before adding prose

Prompt repair usually proceeds by accretion. When an output fails, users append prohibitions, clarifications, examples, emphatic formatting, or additional context. Long prompts become geological deposits of unresolved causality. Delta debugging suggests an opposite discipline. Zeller and Hildebrandt developed procedures that minimize failure-inducing inputs until no further component can be removed without losing the failure [10].

For prompts, the analogous object is a *minimal failing prompt*. Decompose the instruction into clauses, examples, contextual blocks, formatting constraints, or other removable units; repeatedly delete subsets; rerun the model; and retain reductions that preserve the failure with sufficient frequency. The result is not necessarily a better production prompt. It is a stronger research object because it isolates what is sufficient to reproduce the behavior.

Stochastic generation complicates the Boolean oracle assumed by classical delta debugging. A prompt may fail three times in ten rather than always. That does not make reduction useless; it changes the oracle into an estimated failure probability and requires repeated trials. The practice is still valuable because it reverses the default bias toward verbosity. Before "fixing" a prompt by adding language, determine whether the failure can be made smaller.

4.2 Specify relations when exact outputs are unknowable

Generative artifacts often lack a single correct answer. Metamorphic testing was designed for a related testing problem: cases in which the correct output for each input is difficult or impossible to specify directly. Instead, one specifies relations that should hold across transformed inputs and outputs [11].

This maps unusually well onto generative work. If the camera angle changes, identity should remain. If season changes, house geometry should remain. If an object is removed, unrelated objects should not disappear. If a prompt is translated, a substantive requirement may be expected to persist. The executable specification becomes a triple:

$$(T, R, P),$$

where T transforms prompt P into P', and R states the required relation between generated outputs $G(P)$ and $G(P')$.

This is more powerful than a static instruction because it tells the system how to try to falsify the requirement. It also admits a crucial distinction: a pair of outputs can satisfy the relation while both being poor artifacts. Metamorphic relations supplement direct evaluation; they do not eliminate it.

4.3 Make invariants generate their own attacks

QuickCheck provides a complementary move. Property-based testing expresses program properties and automatically evaluates them over generated test inputs [12]. The prompt-practice analogue is a *generative invariant*: not merely "character identity persists," but a property bundled with a generator of transformations likely to threaten that identity—camera, lighting, age, clothing, distance, weather, pose—and an evaluator that checks persistence.

This changes the status of a successful correction. Instead of adding "always preserve the same character" to the next prompt and trusting it, the system can compile that statement into a continuing adversary. A mature prompt field would therefore contain properties that manufacture new attempts to break themselves.

This is especially important because models change. A prompt that "works" on one version is a historical fact, not a permanent contract. Executable invariants can be rerun after model updates, tool changes, or prompt refactoring. The genealogy becomes a regression suite.

4.4 Choose the next prompt for information, not beauty

Active learning offers a fourth operation. Query by Committee selects queries on which competing hypotheses disagree, so the next labeled example is maximally informative about which hypothesis is plausible [13]. Applied methodologically to prompt debugging, the user first writes multiple explanations of a failure, predicts what each would imply under several probe prompts, and executes the probe that most sharply separates them.

Suppose a character disappears when the camera moves. Candidate explanations include underspecified persistence, occlusion language, composition instability, excessive context, or a stochastic branch. Rephrasing until the result improves does not discriminate among these accounts. A diagnostic prompt does. The objective of that generation is epistemic, not productive.

This produces a useful division of labor among the techniques:

Operation	**Question**
Delta reduction	What is minimally sufficient to reproduce the failure?
Diagnostic disagreement	Which next experiment best separates competing explanations?
Metamorphic testing	What relation must hold across controlled changes?
Property generation	What family of cases should continuously attack an invariant?

Together these operations transform correction from prose accumulation into experimental specification.

5 The theory trace

There remains a problem that none of these testing techniques solves by itself: transfer. A final prompt can run successfully while being difficult for another person—or the same person months later—to modify safely.

Naur's "Programming as Theory Building" is useful here because it distinguishes program text from the understanding programmers build about the matters the program handles [14]. Possessing the

text is not equivalent to possessing the theory needed for intelligent modification. Applied carefully to prompts, this suggests that a prompt repository can preserve runnable artifacts while losing the reasons that make those artifacts maintainable.

A correction genealogy can serve as a partial *theory trace*. It cannot capture the entirety of a practitioner's tacit understanding, and it should not pretend to. But it can preserve evidence that ordinary version history omits:

- the output that first exposed a failure;
- the criterion that was added because of that failure;
- competing explanations that were rejected;
- a minimal reproducer;
- the metamorphic relation that a correction is intended to protect;
- cases in which an invariant was deliberately weakened;
- branches that were abandoned and why;
- model or tool versions against which the behavior was observed.

The difference is visible in two version histories. A conventional history says $P_0 \to P_1 \to P_2$. A theory trace says $(P_0, A_0, J_0) \xrightarrow{R_1} P_1$, where R_1 records why the change occurred. The latter can support a future operator asking whether a clause is still necessary or whether removing it resurrects the failure that created it.

This yields a concrete empirical prediction. Give one group only the mature final prompt, a second group the prompt plus static documentation, and a third group the prompt plus its correction genealogy. Ask all three to adapt it to a novel requirement. If genealogy recipients introduce fewer regressions, remove obsolete constraints more safely, or explain failures more accurately, the theory trace has measurable transfer value.

6 Unmarked holes and the changing evaluator

The conjunction of the preceding ideas identifies what may actually be distinctive about current natural-language generative practice. None of its components is wholly unprecedented. The important configuration is:

1. **Semantically underdetermined input can execute.** The user does not need to enumerate every variable required to produce an artifact.
2. **Missing variables may be unmarked.** The system chooses values for consequential dimensions that the user did not explicitly represent as open questions.
3. **The artifact makes omissions visible.** A generated result can reveal a requirement by violating it.
4. **The evaluator can update.** The user may change or newly articulate criteria after inspecting the artifact.
5. **Corrections can become executable tests.** Failures, invariants, and competing explanations can be preserved as procedures rather than prose reminders.
6. **The lineage can be replayed.** A future system can inspect why the present specification exists and test whether its constraints remain necessary.

This is what I mean by *deferred formalization*: execution begins before the consequential specification has been completely represented, and subsequent artifacts participate in making that specification explicit. The term should not be used to claim that formalization itself is unnecessary. On the

contrary, the loop is valuable precisely because it can progressively manufacture more formal objects: counterexamples, invariants, discriminating tests, failure reducers, and provenance records.

The new unit is therefore not "natural language" as such. Natural language is only the first surface. The operational object is a lineage in which some parts remain prose while others harden into tests.

7 Failure modes of the genealogy

Treating the correction genealogy as the program introduces its own risks.

7.1 Accretion without causality

A genealogy can preserve every revision while still failing to explain any of them. Chat history is not automatically a theory trace. The system needs explicit links among observation, hypothesis, test, and change. Delta reduction is valuable partly because it prevents the archive from equating length with knowledge.

7.2 The evaluator can drift

A mutable oracle is powerful only if changes in criteria remain inspectable. Some changes represent genuine specification discovery; others are transient preference shifts. The archive should therefore record whether a new criterion was *previously implicit*, *newly discovered*, *deliberately changed*, or merely *uncertain*. This is a research annotation, not a ground-truth label.

7.3 Memory can become an attractor

Preserving all history is not identical to exposing all history at every execution. The open-ended VLM work described earlier provides a warning that more context can change exploration in non-monotonic ways [5]. A good archive should therefore separate *retention* from *retrieval*. Keep the evidence; decide selectively what the current generation sees.

7.4 Tests can overfit yesterday's model

A property suite can stabilize behavior while also freezing accidental quirks. Some prompt constraints compensate for a particular model version rather than express durable intent. Each invariant therefore needs provenance: *user value*, *task requirement*, *model compensation*, or *unknown*. Migration to a new model should rerun the suite and invite deletion of constraints whose underlying failure no longer exists.

7.5 Open-endedness can dissolve the task

Novelty search and POET make strong provocations, but indiscriminate objective mutation can become ordinary drift. A useful system needs lineage-preserving criteria for when a reframing remains a descendant of the original problem. The research question is not whether goals may change; it is how to distinguish productive reframing from abandonment.

8 A protocol for prompt work

The argument can be condensed into a practical protocol. When a generative task matters enough to maintain, the operator should preserve not just a prompt but an experimental field.

1. **Run.** Execute the current description and preserve the exact output and model/tool context.
2. **Name the difference.** State what failed, surprised, or became newly salient.
3. **Classify the change.** Is this an output error, a newly discovered specification, a preference change, or a model-specific compensation?
4. **Reduce when possible.** Before adding prose, isolate a minimal reproducer of the failure.
5. **Compete explanations.** Maintain more than one causal account and choose diagnostic generations that distinguish them.
6. **Promote stable relations.** Convert accepted constraints into metamorphic relations or property generators.
7. **Preserve branches.** Do not erase strange stepping stones merely because they were not selected.
8. **Separate archive from context.** Preserve full history while selectively retrieving what the next execution needs.
9. **Record reasons.** Every consequential constraint should point back to the evidence that caused it.
10. **Re-test after change.** Model updates, tool changes, and refactoring should rerun the accumulated specification.

This protocol makes a subtle shift in the role of the user. The user is not simply authoring instructions. They are constructing an experimental apparatus around a generative system. Prompt craft becomes closer to a laboratory practice: formulate a move, produce an observation, discriminate hypotheses, stabilize an invariant, and keep enough provenance for another investigator to reproduce the reasoning.

9 Research agenda

Several empirical tests would determine whether the correction genealogy is genuinely the better unit.

Final prompt versus genealogy transfer. Measure whether people adapting a mature workflow perform better with only the final prompt, with static documentation, or with a structured correction genealogy.

Unmarked-hole inventory. Compare formal partial specifications with natural-language descriptions and count consequential variables introduced by the generator but absent from the input. Track which variables become explicit only after the first artifact.

Fixed versus mutating evaluation. Pre-register a rubric, run iterative generation, and record every criterion added, removed, or reweighted. Re-evaluate the final artifact under the initial rubric to test how much acceptance depends on specification change.

Prompt accretion versus failure minimization. Compare ordinary "add another instruction" debugging with stochastic delta reduction. Measure prompt length, failure recurrence, and explanatory accuracy.

Static rule versus executable property. Preserve the same constraint either as prose or as a perturbation generator plus evaluator. After model migration, compare regression detection.

Goal search versus stepping-stone preservation. Compare branches selected solely for target resemblance with branches that deliberately preserve novelty. Trace whether final high-value artifacts descend through temporary decreases in target similarity.

These experiments would also expose where the theory fails. It may turn out that some domains are well served by isolated prompts because the task is stable and failures are obvious. The correction genealogy should not become a universal metaphysics of interaction. Its scope is strongest where outputs are generative, specifications are incomplete, failures are legible, and users iteratively discover criteria through inspection.

10 Conclusion

The prompt is an unusually visible artifact, but visibility should not be mistaken for computational or epistemic primacy. Early prompt communities already practiced something richer than instruction writing: they searched, selected, branched, hoarded, remixed, randomized, and learned through repeated execution [1]. Older traditions show that none of the components—reflective conversation, partial programs, human evaluation, counterexample-guided revision—belongs uniquely to generative AI [7, 8, 2, 9]. That genealogy narrows rather than weakens the contemporary claim.

What current generative systems make unusually accessible is execution under semantically unmarked incompleteness. They can produce an artifact before the user has represented all consequential variables; the artifact can make an omission visible; the evaluator can update; and the correction can be promoted into an executable test. Once this happens repeatedly, the specification is no longer located in one prompt. It is distributed across the lineage that explains what was tried, what happened, what mattered, and what must continue to hold.

The practical implication is simple. Stop treating the mature prompt as the thing worth archiving. Preserve the evidence that made it mature. A correction genealogy can become a portable specification: not a transcript of everything that happened, but a reproducible field of failures, tests, branches, reasons, and invariants. The prompt is one state in that field. The program is the work that can continue.

Assembly note

This manuscript is a derived interpretation, not archival evidence. Its source map in the accompanying package traces substantive claims through research cards to cited sources. The exact assembly instrument is preserved at `_PROMPTS/ANCIENT_MASTER_SLIPCASE_v15.5-AM.txt`. The manuscript should be read as a current wager that can be revised without altering the source cards from which it was assembled.

References

[1] Zhoujun Sun, Watson Hartsoe, and Tommy Ottolin. *Sculptors of Noise: Control in Midjourney AI Art Community*. Georgia Institute of Technology, 2022.

[2] Hideyuki Takagi. "Interactive Evolutionary Computation: Fusion of the Capabilities of EC Optimization and Human Evaluation." *Proceedings of the IEEE* 89(9), 1275–1296, 2001. doi:10.1109/5.949485.

[3] Joel Lehman and Kenneth O. Stanley. "Abandoning Objectives: Evolution Through the Search for Novelty Alone." *Evolutionary Computation* 19(2), 189–223, 2011. doi:10.1162/EVCO_a_00025.

[4] Jimmy Secretan et al. "Picbreeder: Evolving Pictures Collaboratively Online." *CHI 2008*, 1759–1768, 2008. doi:10.1145/1357054.1357328.

[5] Sam Earle et al. "In Search of the Ingredients of Open-Endedness: Replicating Picbreeder with Large Vision-Language Models." GECCO 2026. arXiv:2605.23908.

[6] Rui Wang, Joel Lehman, Jeff Clune, and Kenneth O. Stanley. "Paired Open-Ended Trailblazer (POET)." arXiv:1901.01753, 2019.

[7] Donald A. Schön. "Designing as Reflective Conversation with the Materials of a Design Situation." *Research in Engineering Design* 3, 131–147, 1992. doi:10.1007/BF01580516.

[8] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD dissertation, University of California, Berkeley, 2008.

[9] Susmit Jha and Sanjit A. Seshia. "A Theory of Formal Synthesis via Inductive Learning." arXiv:1505.03953, 2015.

[10] Andreas Zeller and Ralf Hildebrandt. "Simplifying and Isolating Failure-Inducing Input." *IEEE Transactions on Software Engineering* 28(2), 183–200, 2002. doi:10.1109/32.988498.

[11] T. Y. Chen, S. C. Cheung, and S. M. Yiu. "Metamorphic Testing: A New Approach for Generating Next Test Cases." HKUST-CS98-01, 1998.

[12] Koen Claessen and John Hughes. "QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs." *ICFP 2000*, 268–279. doi:10.1145/351240.351266.

[13] H. S. Seung, M. Opper, and H. Sompolinsky. "Query by Committee." *COLT 1992*, 287–294. doi:10.1145/130385.130417.

[14] Peter Naur. "Programming as Theory Building." *Microprocessing and Microprogramming* 15(5), 253–261, 1985. doi:10.1016/0165-6074(85)90032-8.